本科生计算复杂性理论报告

题　目：　第一次编程作业

组长学号	**2333xxxx**
组长姓名	刘昊
组员**1**学号	**23336203**
组员**1**姓名	任意嵘
组员**2**学号	**2333xxxx**
组员**2**姓名	彭怡萱
专业名称	计算机科学与技术
任课教师	冯剑琳
实验时间	**2026**年**3**月**23**日—**4**月**3**日

目　录

一、实验目的与要求 …… 1

二、模块设计思路 …… 1

三、关键代码展示 …… 2

四、运行结果展示 …… 4

一、 实验目的与要求

本次编程作业要求同学们以小组为单位，编写 Python 程序实现 `tm2.ppt` 中的图灵机编码、多带图灵机以及通用图灵机，并以两数加法图灵机 ADD 为测试用例进行测试。这三个部分的实现并不割裂，应当在同一个 Python 程序中组合完成整个工作流。

具体的核心要求如下：

1) **图灵机编码模块**

• 读取定义图灵机的 JSON 文件，并将其转换为二进制编码。

• 按照特定的规则，将元素映射为正整数：例如将图灵机的状态 q_i、符号 X_j 和移动方向 D_m 等映射之后，用 0 的数量表示大小，元素间由 1 隔开，最终转为由 0 和 1 构成的二进制字符串。

2) **多带图灵机模块**

• 实现一个多带图灵机模拟器，模拟实现多条磁带、多磁头的独立工作以及状态转移操作。

• **实时展示运行过程**：在多带图灵机的运行过程中，需要在屏幕上实时格式化打印出当前状态、各条磁带的内容（含边界分隔符等），以及各个磁头的位置。

• **执行模式控制**：程序应当支持两种展示模式：

1) **交互式**：图灵机每一步状态转移后，等待用户敲击空格键后继续下一步运行。

2) **自动式**：图灵机每一步状态转移后，等待可配置的固定时间（例如 0.5 秒）后自动继续下一步运行。

3) **通用图灵机（UTM）模块**

• 基于多带图灵机的代码，实现通用图灵机模拟。

• 设定 UTM 初始状态下第一条磁带（Tape 1）包含完整输入 $M111w$。其中 M 为通过编码模块获得的被模拟图灵机编码，w 为该图灵机的输入字符串。

• 模拟时，Tape 2 专门用于存放模拟图灵机 M 的纸带并标记 M 的磁头位置；Tape 3 专门用于保存 M 当前的状态。

• 为了简化实现，允许假定输入给通用图灵机的代码总是合法的，省略并跳过初始的合法性检查步骤（Step 1）。

• 除了多带图灵机常规的纸带实时打印外，还需在每次状态切换时根据 `tm2.ppt` 中的步骤，打印出**当前所处的运行阶段的完整说明文本**。

4) **用例测试**

• 构造 `tm more examples.pdf` 中两数加法图灵机 ADD 的 JSON 定义文件作为输入。

• 根据 JSON 里 `input` 字段（如 `["00", "000"]`，即分别代表输入 2 和 3 的一进制编码），执行 UTM 运转测试。

二、 模块设计思路

(1) 图灵机编码模块

图灵机编码模块的主要功能是读取表示图灵机的 JSON 定义文件，解析出其中的状态及状态转移函数，将元素正确映射为独立的正整数，最终利用一进制（Unary）策略

将图灵机的行为转换为纯粹的由 `0` 和 `1` 构成的二进制字符串。为保证编码过程的确定性与稳健性，具体设计思路展开如下：

1) **JSON 解析、校验与归一化（Normalization）：**

• **容错校验：**程序读取 JSON 后，提取其中的字典结构。遍历每一个状态及其挂载的转移条件，严格校验是否存在 `write`, `move`, `nextState` 等核心字段。

• **字段归一化：**对于存在多种书写偏好的符号，进行底层归化以消除歧义。例如，将空白符的不同别名（如 `""`, `" "`, `"blank"` 等）统一转化为标准的单空格 `" "`；将移动方向（如大写的 `"L"`, `"Left"` 等）全部统一映射为内部标志 `left`, `right`、`stay` 以供直接调用。

2) **状态（States）的无碰撞 ID 映射：**

• **强制映射分配：**利用正则表达式 `^q([1-9]\d*)$` 提取符合标准命名的状态数字后缀，将其直接作为该图灵机状态的标号（例如，提取 `q1` 中的 1 并分配极值给它）。为了防止 `q1` 的对应数字 1 被其他非标准状态误占，代码中会专门维护一个已用数字池（`used_ids`）。

• **非标准状态的字典序填补：**如果存在其它命名规则的标志状态（如 `start`, `accept` 等），将所有状态名称置于字典序下排序后，以递增遍历的方式为其寻找当前未被标记占用的最小正整数。通过两步解耦的设计彻底避免了标号碰撞。

3) **符号与方向的 ID 固定分配：**

• 根据实验规范对基础字符进行强绑定：数值 `0` 锁定对应 ID 1；数值 `1` 锁定对应 ID 2；空白符（`blank`）锁定为 ID 3。其余被图灵机使用的未知工作字符集合也将基于字典序从 4 开始向上顺序累加。

• 移动方向采用静态字典约束，按规定映射：`left`$\rightarrow 1$, `right`$\rightarrow 2$, `stay`$\rightarrow 3$。

4) **状态转移列表的稳健排序：**为了保证一台确定的图灵机会产生截然不变的编码字符串，必须在遍历所有状态转移生成代码之前对其做全排。模块采用了一个五维元组作为图灵机转移的排序 KEY：`((`当前态`ID), (`读符号`ID), (`下状态`ID), (`写符号`ID), (`移动方向`ID))`。

5) **一进制拼接与组合连结：**排序完成后，针对五步数组中的每一个正整数编号 N，利用函数生成长度为 N 的连续零串（例如 3 生成 `000`）。同一转移式中的五项动作内用单个 `1` 隔断；不同转移表达式之间最后连结时插入 `11` 成为全局统一编码。

(2) 多带图灵机模块（待完成）

（此处为多带图灵机的设计思路，当前暂时预留）

(3) 通用图灵机模块（待完成）

（此处为通用图灵机的设计思路，当前暂时预留）

三、 关键代码展示

(1) 图灵机编码：转移序列的确定性多键排序

利用 Python `sorted` 内置方法与多重映射，保证编码输出无论 JSON 顺序如何都恒定一致：

```python
# src/modules/encoding.py (片段)
ordered_transitions = sorted(
    transitions,
    key=lambda transition: (
        state_ids[transition.current_state],
        symbol_ids[transition.read_symbol],
        state_ids[transition.next_state],
        symbol_ids[transition.write_symbol],
        DIRECTION_IDS[transition.move],
    ),
)
```

(2) 图灵机编码：防冲突的状态 ID 映射逻辑

优先预留 qN 的固有位值，后续按字典序填补空挡寻找正整数。

```python
def build_state_ids(transitions: List[Transition]) -> Dict[str, int]:
    state_names = { # 通过集合推导式去重合并起点与终点状态
        transition.current_state for transition in transitions
    } | {transition.next_state for transition in transitions}

    state_ids: Dict[str, int] = {}
    used_ids: Dict[int, str] = {}

    # 阶段1：严格提取规范化 qN 中的数字 N 进行挂载
    for state_name in sorted(state_names):
        match = STATE_NAME_PATTERN.fullmatch(state_name)
        if not match:
            continue
        state_id = int(match.group(1)) # 取出数字
        # 异常校验已省略
        state_ids[state_name] = state_id
        used_ids[state_id] = state_name

    # 阶段2：对剩余的一般化字符串补充映射
    next_available = 1
    for state_name in sorted(state_names):
        if state_name in state_ids:
            continue
        while next_available in used_ids: # 探查下一个空置数值
            next_available += 1
        state_ids[state_name] = next_available
        used_ids[next_available] = state_name
        next_available += 1

    return state_ids
```

(3) 图灵机编码：一进制翻译核心封装

将单个转移元组格式化为一进制序列并将全集编码串联起来：

```python
def encode_transition(
    transition: Transition,
    state_ids: Dict[str, int],
    symbol_ids: Dict[str, int],
) -> str:
    # 逐项转换为 N 个 0 表示的等效一进制数组
    parts = [
        unary_encode(state_ids[transition.current_state]),
        unary_encode(symbol_ids[transition.read_symbol]),
        unary_encode(state_ids[transition.next_state]),
        unary_encode(symbol_ids[transition.write_symbol]),
        unary_encode(DIRECTION_IDS[transition.move]), # left:1, right:2
    ]
    return "1".join(parts) # 元素内部只用一个 1 作为隔离符

# 对全局数组应用并聚合
return EncodingResult(
    encoding="11".join(item.encoding for item in encoded_transitions),
        # 语句间夹杂 11
    # ... 其他结构
)
```

(4) 多带图灵机与通用图灵机核心代码（待完成）

（此处为待完成的代码展示段落，当前暂时预留）

四、 运行结果展示

(1) 图灵机编码模块测试

运行交互式入口程序并选择对应功能模块：

```
请选择功能：
1. 列出 test 目录下的共享 JSON 样例
2. 对选定样例执行图灵机编码
3. 预留：多带图灵机模块
4. 预留：通用图灵机模块
5. 退出程序
请输入编号：2
请选择测试样例：
1. sample_tm.json (test/sample_tm.json)
请输入编号：1
```

对应的输出结果如下，得到按照给定规则的一连串 0 与 1 的正确编码结果（如图 1 所示）：

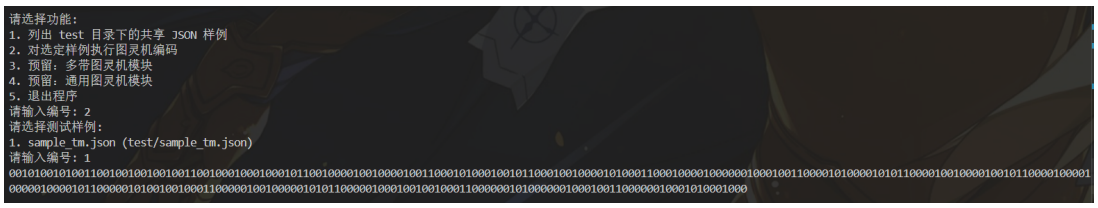

图1 图灵机编码模块交互与结果输出截屏

(2) 多带与通用图灵机功能测试（待完成）

（模拟演示结果，当前暂时预留）